

Days 1-2: Python Fundamentals for Java Developers

Day 1: Python Syntax & Basic Data Types

Setup (15 minutes)

1. **Install Python:** Download Python 3.11+ from python.org
2. **IDE Options:**
 - **VS Code** with Python extension (recommended)
 - **PyCharm Community Edition**
 - **IntelliJ IDEA** with Python plugin (familiar territory!)

Core Syntax Differences (30 minutes)

Variables & Types

python

```
# Python - Dynamic typing
name = "Alice" ..... # String
age = 25 ..... # Integer
height = 5.8 ..... # Float
is_student = True ..... # Boolean
```

```
# Java equivalent:
# String name = "Alice";
# int age = 25;
# double height = 5.8;
# boolean isStudent = true;
```

Print Statements

python

```
# Python
print("Hello, World!")
print(f"Name: {name}, Age: {age}") # f-strings (Like String.format)

# Java equivalent:
# System.out.println("Hello, World!");
# System.out.printf("Name: %s, Age: %d\n", name, age);
```

Comments

python

```
# Single Line comment
"""
Multi-line comment
(like /* */ in Java)
"""
```

Collections - The Big Difference (45 minutes)

Lists (like ArrayList)

python

```
# Python Lists
numbers = [1, 2, 3, 4, 5]
mixed_list = [1, "hello", 3.14, True] # Can mix types!

# Common operations
numbers.append(6) # add()
numbers.insert(0, 0) # add(index, element)
numbers.remove(3) # remove(Object)
first = numbers[0] # get(0)
numbers[1] = 10 # set(1, 10)
length = len(numbers) # size()

# Java equivalent would need generics and more verbose syntax
```

Dictionaries (like HashMap)

python

```
# Python Dictionary
student = {
    "name": "Alice",
    "age": 25,
    "grades": [85, 90, 78]
}

# Access and modify
name = student["name"] # get("name")
student["age"] = 26 # put("age", 26)
student["major"] = "CS" # put("major", "CS")

# Safe access (Like Optional in Java)
major = student.get("major", "Undeclared") # getOrDefault
```

Sets (like HashSet)

python

```
# Python Set
unique_numbers = {1, 2, 3, 4, 5}
unique_numbers.add(6) ..... # add()
unique_numbers.remove(1) ..... # remove()
contains_3 = 3 in unique_numbers .. # contains()
```

Control Flow (30 minutes)

If Statements

python

```
# Python - note the colon and indentation!
if age >= 18:
.... print("Adult")
elif age >= 13:
.... print("Teenager")
else:
.... print("Child")

# Java equivalent uses braces instead of indentation
```

Loops

python

```
# For Loop - much simpler than Java!
for i in range(5): ..... # for(int i = 0; i < 5; i++)
.... print(i)

for name in ["Alice", "Bob", "Charlie"]: .. # Enhanced for Loop
.... print(f"Hello, {name}")

# While Loop
count = 0
while count < 5:
.... print(count)
.... count += 1
```

Practice Exercises - Day 1

Exercise 1: Basic Variables Create a program that stores your personal information and prints it:

python

```
# TODO: Create variables for name, age, city, and favorite_languages (List)  
# Print them in a formatted way
```

Exercise 2: List Operations

python

```
# TODO: Create a List of numbers 1-10  
# Add 11 and 12 to the end  
# Remove the number 5  
# Print the final List and its Length
```

Exercise 3: Dictionary Practice

python

```
# TODO: Create a dictionary representing a book with title, author, pages, and year  
# Add a new field "genre"  
# Print each field with a Label
```

Day 2: Functions & More Data Structures

Functions - More Flexible Than Java Methods (45 minutes)

Basic Function Syntax

python

```
# Python function  
def greet(name):  
    .... return f"Hello, {name}!"  
  
# Call it  
message = greet("Alice")  
print(message)  
  
# Java equivalent:  
# public static String greet(String name) {  
# .... return "Hello, " + name + "!";  
# }
```

Default Parameters (Not in Java!)

python

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Alice")) ..... # "Hello, Alice!"
print(greet("Bob", "Hi")) ..... # "Hi, Bob!"
```

Variable Arguments (*args, **kwargs)

python

```
def sum_numbers(*args): .. # Like varargs in Java
.... return sum(args)

result = sum_numbers(1, 2, 3, 4, 5) .. # 15

def create_student(**kwargs): .. # Named arguments
.... return kwargs

student = create_student(name="Alice", age=25, major="CS")
# Returns: {'name': 'Alice', 'age': 25, 'major': 'CS'}
```

Advanced Data Structure Operations (30 minutes)

List Comprehensions (Very Pythonic!)

python

```
# Instead of:
squares = []
for i in range(10):
.... squares.append(i * i)

# Write this:
squares = [i * i for i in range(10)]

# With conditions:
even_squares = [i * i for i in range(10) if i % 2 == 0]

# Java equivalent would need Stream API:
# List<Integer> squares = IntStream.range(0, 10)
# .... .map(i -> i * i)
# .... .boxed()
# .... .collect(Collectors.toList());
```

Dictionary Comprehensions

python

```
# Create a dictionary of squares
squares_dict = {i: i * i for i in range(5)}
# Result: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Tuple (Immutable Lists)

python

```
# Tuples are like immutable lists
coordinates = (10, 20)
name_age = ("Alice", 25)

# Unpacking (multiple assignment)
x, y = coordinates
name, age = name_age

# Great for returning multiple values
def get_name_age():
    ... return "Alice", 25

name, age = get_name_age()
```

String Operations (30 minutes)

python

```
# String methods (similar to Java)
text = "... Hello, World! ..."
print(text.lower()) ... # toLowerCase()
print(text.upper()) ... # toUpperCase()
print(text.strip()) ... # trim()
print(text.replace("Hello", "Hi")) ... # replace()

# Splitting and joining
words = text.split(",") ... # split()
joined = "-".join(words) ... # String.join() but different syntax

# F-strings (modern string formatting)
name = "Alice"
age = 25
message = f"My name is {name} and I'm {age} years old"
# Better than: "My name is " + name + " and I'm " + age + " years old"
```

Practice Exercises - Day 2

Exercise 1: Function Practice

python

TODO: Write a function that takes a List of numbers and returns:

- The sum

- The average

- The maximum

- The minimum

Use tuple unpacking to return multiple values

```
def analyze_numbers(numbers):
```

```
..... # Your code here
```

```
..... pass
```

Test with: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Exercise 2: List Comprehensions

python

TODO: Given a List of words, create:

1. A List of word length